

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 633 — Machine Learning

Estimation, generalisation, and evaluation — with honest measurement treated as the subject rather than the last chapter

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 6 (with CSCE 636 and CSCE 669)

Prerequisites: CSCE 629 Analysis of Algorithms; linear algebra and probability; UC MATH 600 and UC LINA 600 if either is rusty

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A model trained on data you did not choose, with a held-out evaluation you designed before seeing the result, an honest baseline comparison, and a written account of what it will do when deployed

Course description

Machine learning is the first subject in this program where **the thing you are optimising is not the thing you want**. Every other course had a specification: the hull must be convex, the compiler must preserve meaning, the renderer must solve the rendering equation. Here you have a sample, a loss function that approximates a goal, and a hope that performance on data you have predicts performance on data you have not.

That hope is the entire subject. Module 03 states it precisely as the bias–variance decomposition and Module 04 as the generalisation gap, and essentially every technique in the course — regularisation, cross-validation, ensembling, early stopping — is a way of managing the distance between training performance and the performance you actually care about.

The second theme is that **evaluation is harder than training**, and is where almost all real failures live. A model that reports 99% accuracy on a dataset where 99% of examples are negative has learned nothing. A model evaluated on data that leaked from its own training set reports a number that means nothing. **Module 05 is about metrics and Module 12 is about the ways an evaluation lies**, and both are placed early enough to be used rather than late enough to be skipped.

The emphasis throughout is on **what the model will do when it meets data drawn from a different distribution than it was trained on**, because it will. **Module 13 argues that the deployment questions — drift, feedback loops, and who is harmed when it is wrong — are engineering questions with technical answers**, not an ethics appendix.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State the supervised learning problem precisely and say what is assumed.
- 2 Derive and implement linear and logistic regression from the loss.
- 3 Decompose error into bias, variance, and noise, and act on it.
- 4 Apply regularisation and explain what it buys.
- 5 Choose and compute evaluation metrics appropriate to the problem.
- 6 Build and tune tree ensembles, and say why they win on tabular data.
- 7 Apply kernels and explain the representer theorem's consequence.
- 8 Apply unsupervised methods and state what their output means.
- 9 Diagnose leakage, imbalance, and distribution shift.
- 10 Report a result honestly, including the baseline it beat.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What Learning From Data Means	What exactly is being assumed when a model is trained?
2	Linear Models	Where does the fitting rule come from?
3	Bias, Variance, and Capacity	Why does a model that fits the training data perfectly do badly?
4	Regularisation and Model Selection	How do you control overfitting and pick between models?
5	Evaluation and Metrics	What does 'the model is good' actually mean?
6	Trees and Ensembles	Why do gradient-boosted trees beat neural networks on spreadsheets?
7	Kernels and Margins	How do you fit a nonlinear model with a linear method?
8	Clustering and Density	What does it mean to find groups in unlabelled data?
9	Dimensionality Reduction	How do you compress features without losing what matters?
10	Probability and Uncertainty	What should a model say when it does not know?
11	Features, Data, and Labels	What should you spend your time on?
12	How Evaluations Lie	Why was the model worse in production than in the notebook?

Wk	Module	Core question it answers
13	Deployment	What happens after the model is good enough?

Projects

Project 1 — A model, and an honest evaluation of it (after Module 7)

Take a dataset you did not choose and build a supervised model for it — with the evaluation protocol written down *before* you see any results.

Requirements

- **A written evaluation protocol, dated, committed before the first model is trained.** Which split, which metric, which baseline.
- **A trivial baseline** — majority class, or the mean, or one feature — that every later model must beat.
- A proper train / validation / test split, with the test set used **exactly once**.
- **Learning curves** showing training and validation error against training set size and against model complexity.
- A bias–variance diagnosis stating which one limits you and what you did about it.
- **A leakage audit:** for each feature, could it have been known at prediction time?

Done means

- **The test set was evaluated once and the number is reported, whatever it was.** Re-running it after tuning invalidates it, and saying so is part of the exercise.
- **Learning curves with the diagnosis written on them** — high bias, high variance, or neither.
- **The trivial baseline's score, next to yours.** If the gap is small, that is the finding.
- **A list of what you tried that did not work.** This is graded, and omitting it is the normal dishonesty of the field.

Project 2 — Harder data, and what happens next (after Module 12)

Take the model further on data that is imbalanced, shifted, or small — and say what it will do in deployment.

Requirements

- **A dataset with real class imbalance, missing data, or a temporal structure,** handled explicitly rather than ignored.
- An ensemble, with the individual models' scores reported alongside it.
- **Calibration measured and reported,** not just discrimination — a reliability diagram.
- **A distribution-shift experiment:** train on one slice, test on another, and report the degradation.
- An error analysis: a sample of the mistakes, categorised.
- Feature importance or another account of what the model is using.

Done means

- **A reliability diagram, with a statement of whether the probabilities mean anything.**
- **The measured drop under distribution shift**, with a stated expectation of how fast it would degrade in production.
- **An error analysis naming the categories of mistake** and which are acceptable.
- **A deployment statement:** who is affected when it is wrong, what the feedback loop is, and what would make you retrain it.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Stanford CS229 — Machine Learning (free lectures and notes)	The primary source. Andrew Ng's lecture notes are the best free derivation-level treatment of Modules 02 through 08, and the problem sets are worth doing.
James, Witten, Hastie & Tibshirani — An Introduction to Statistical Learning (free PDF)	Free, and the right level for this course. Strong on the bias–variance material of Module 03 and on resampling. The R and Python editions are both free.
Hastie, Tibshirani & Friedman — The Elements of Statistical Learning (free PDF)	The deeper companion to the above, also free. Use it for the derivations the first book states without proof.
scikit-learn — the user guide (free)	Unusually good documentation — the model selection and metrics chapters are genuinely instructional, not merely reference material.

Resource	Role
Google — Rules of Machine Learning (free)	The Module 13 material. Forty-three rules from people who have deployed a great deal of this, and the early ones are about not using machine learning yet.
Kaufman, Rosset & Perlich — Leakage in Data Mining (free)	The Module 12 reference. Worked examples of evaluations that were wrong and the competitions they won.

Tooling

- **Python with NumPy, scikit-learn, and pandas.** Implement the first few algorithms from scratch in NumPy, then use the library — the same pattern as CSCE 735's advice about libraries.
- **A notebook environment, used with discipline.** Notebooks make it very easy to evaluate on data you have already looked at, which is Module 12's central hazard.
- **A fixed random seed, and a results log.** Record every experiment you run, including the ones that failed — otherwise you will rediscover the same dead ends and will overstate your successes.
- **A dataset you did not choose.** The project requires one; picking a clean benchmark you already know the answer on defeats the purpose.
- **A plotting setup,** because learning curves, calibration plots, and confusion matrices are how you diagnose a model and none of them is a single number.
- **Version control for data as well as code.** A result you cannot reproduce because the preprocessing changed is not a result.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.